Mark	9/11

Team name:	B9		
Homework number:	Homework 09		
Due date:	18/11/2025		
Contribution:	No	Partial	Full
Pica Mirko			✅
Prendin Christian			✅
Pignataro Gabriele			✅
Moazenkakhki Mostafa			✅
Yarrabbotula Sunandha			✅
Notes:			

Project name	LED Matrix		
Not done	Partially done (major problems)	Partially done (minor problems)	Completed
			✅

Part 1. Finish the LED matrix (SPI) project.

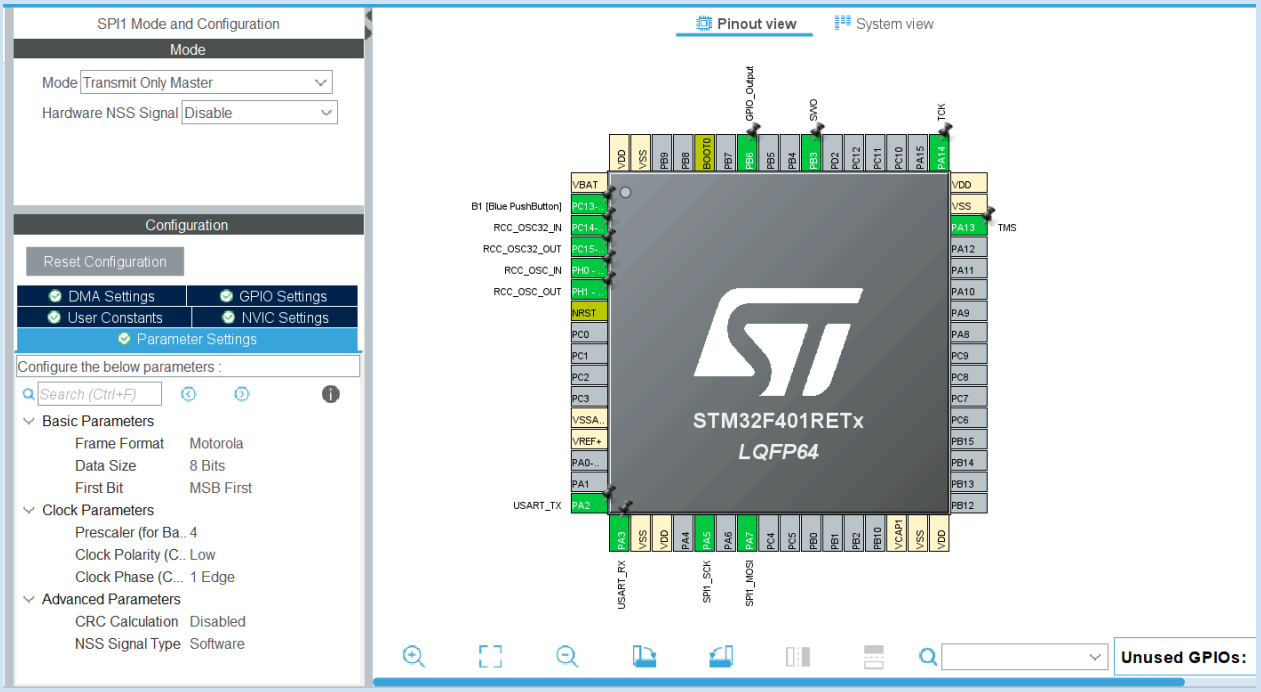
Explanation: The homework was successfully completed. In the following sections, all the steps of the implementation are described in detail.

Step 1: Pin and peripheral configuration in .ioc

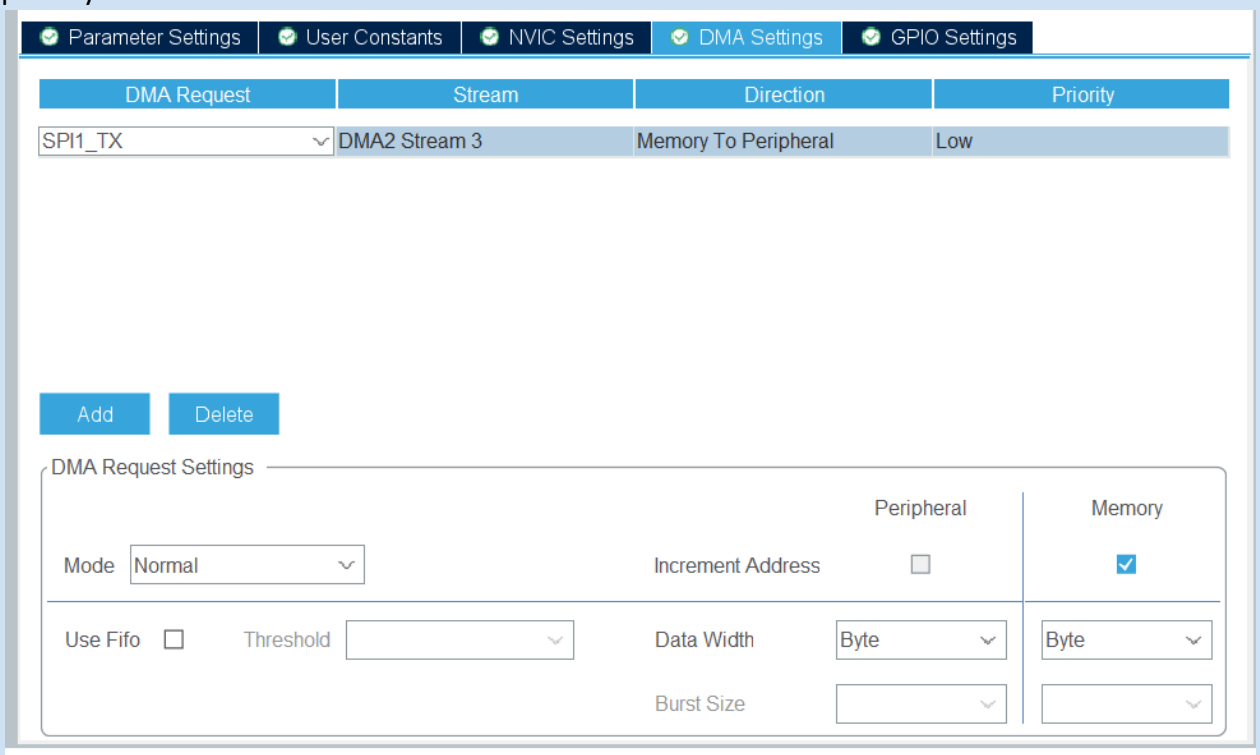
1. SPI

First, **SPI1** was enabled and configured in *Transmit Only Master* mode. In the **Parameter Settings** menu, the only parameter that was modified was the prescaler, which was set to **4**.

The required pins for SPI operation were configured: **PA5** and **PA7** were assigned to SPI1, and **PB6** was used as the **RCLK** signal (see the figures below)



Then, in the **DMA Settings** tab, the **SPI1_TX** DMA request was added in *Normal* mode with *Low* priority.

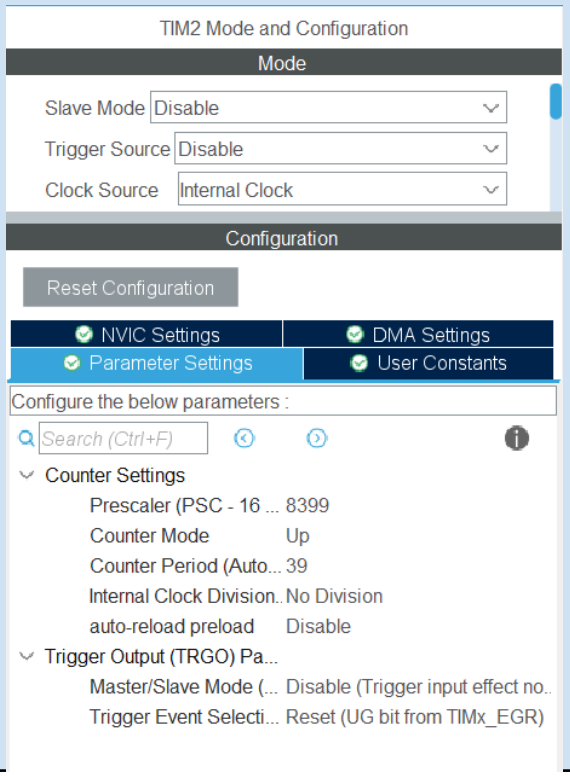


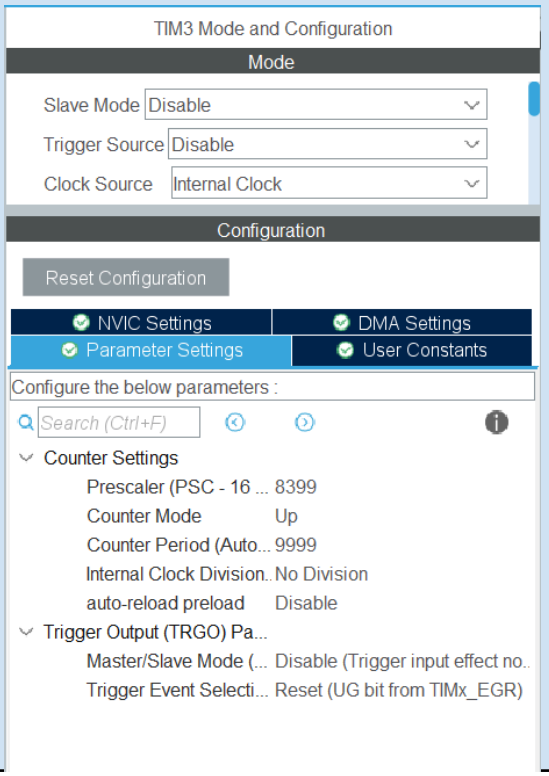
And the **SPI1** interrupt was then enabled in the **NVIC Settings**.

NVIC Interrupt Table	Enabled	Preemption Priority	Sub Priority
SPI1 global interrupt	☑	0	0
DMA2 stream3 global interrupt	☑	0	0

2. TIMER configuration

For this homework, two timers were configured. The first timer (**TIM2**) was set to generate an interrupt every **4 ms** (250 Hz) in order to refresh one column of the LED matrix and start the SPI transmission. The second timer (**TIM3**) was set to generate an interrupt every **1 s** (1 Hz) to change the currently displayed symbol. The global interrupts for both timers were enabled. The prescaler and counter period values used for these configurations are shown in the screenshots below in the row.





After the configuration was completed, the code was generated.

Step 2: Code

In this step, the implementation in `main.c` was considered. The project was implemented to display two letters that are alternated every 1 s. To generate the desired symbols on the 5×7 LED matrix, the provided Excel file was used to derive the row and column bit patterns corresponding to each LED for the characters **B** and **9**. These patterns were then encoded in the firmware as the constant arrays `columns[COL]` and `rows[SYM][COL]`, as shown in the code snippet below

```
/* USER CODE BEGIN PD */
#define COL 5
#define SYM 2
/* USER CODE END PD */

```

```
/* USER CODE BEGIN PV */
const uint8_t columns[COL] = {16, 8, 4, 2, 1};
const uint8_t rows[SYM][COL] = {{127, 73, 73, 73, 54}, {48, 73, 73, 73, 54}}; // B9


volatile int current_column = 0;
volatile int current_symbol = 0;
/* USER CODE END PV */
```

The `HAL_TIM_PeriodElapsedCallback` function is used to handle the timer interrupts. When **TIM2** is triggered, a 2-byte buffer is built with the row pattern and column mask for the current column of the current symbol, and these two bytes are sent to the LED matrix using `HAL_SPI_Transmit_DMA`. In this way, one column of the display is updated at each TIM2 event. When **TIM3** is triggered, the variable `current_symbol` is updated as `(current_symbol + 1) % SYM`, so the displayed character is switched between the stored symbols.

```c
/* USER CODE BEGIN 0 */
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
  if (htim->Instance == TIM2) {
      uint8_t tx_buffer[2] = {
          rows[current_symbol][current_column],
          columns[current_column]
      };
      HAL_SPI_Transmit_DMA(&hspi1, tx_buffer, 2);
  } else if (htim->Instance == TIM3) {
      current_symbol = (current_symbol + 1) % SYM;
  }
}

```

The callback function shown below is executed when the SPI DMA transfer has finished. First, it is verified that the interrupt originates from **SPI1**. Then, pin **PB6** is toggled, generating a latch pulse for the shift registers so that the new row and column data appear on the LED matrix. Finally, the variable `current_column` is updated as `(current_column + 1) % COL`, so that the next refresh will move on to the following column.

```c
void HAL_SPI_TxCpltCallback(SPI_HandleTypeDef *hspi){
    if (hspi->Instance == SPI1) {
        HAL_GPIO_WritePin(GPIOB, GPIO_PIN_6, GPIO_PIN_SET);
        HAL_GPIO_WritePin(GPIOB, GPIO_PIN_6, GPIO_PIN_RESET);

        current_column = (current_column + 1) % COL;
    }
}
/* USER CODE END 0 */

```

Finally, in `main()`, both timers are started with their interrupts enabled.

```c
/* USER CODE BEGIN 2 */
HAL_TIM_Base_Start_IT(&htim2);
HAL_TIM_Base_Start_IT(&htim3);
/* USER CODE END 2 */
```

In conclusion, the LED matrix was correctly controlled using SPI1 with DMA and two hardware timers. The column and row patterns obtained from the Excel file were stored in lookup tables and used to refresh one column every 4 ms, while a second timer periodically changed the active symbol. The use of timer interrupts and the SPI transmit-complete callback allowed the display to be updated without blocking delays, and the final result matched the expected of the characters B and 9.	
Professor comments:	